

Meeting Notes from Pannier Review

Things to Find out Next

- Nominal Schedule vs Accelerated Schedule

Overall Goal

- ☐ Implement 7th/8th E-Axis of robot for controlling the extruder using a RP2040 based microcontroller
 - ☐ Actuator - stepper motor
 - ☐ Comms from robot controller - RTDE
 - ☐ Power - 5.1V, 24V (power the Pi, microcontroller, AND actuator)
 - ☐ Stepper ~ 24V (need datasheet)
 - ☐ Pico takes 12-24V (need datasheet)
 - ☐ Pi (need datasheet)
 - ☐ I/O pins - TBD (UR30 pg 84 for ref)
 - ☐ Language implementation - [Lingua Franca](#); OR Klipper
 - ☐ **Need to trade this**
 - ☐ May need to write libraries or incorporate C++ messages
 - ☐ Location
 - ☐ For posterity, probably end effector, but keep options open
 - ☐ **Trade this**

Electrical

Documentation of

- ☐ Physical layer of communication
- ☐ Which pin serves which function
 - ☐ Comms vs power vs voltage level, etc
 - ☐ Via a diagram, printable packet, etc

Software

- ☐ Lingua Franca implementation (see Canvas feedback) - not MicroPython (see trades)
 - ☐ Slicing up TCP/IP comms at the LF level to send/receive
 - ☐ Need libraries (if they don't exist already, develop them) to
 - ☐ Control microcontroller <-> stepper functionality

- ☐ Parse & route RTDE command (+ handshakes)

Mechanical

- ☐ Dawood should handle
 - ☐ Packaging
 - ☐ Cabling
 - ☐ End effector
 - ☐ Procurement (low lead time parts)

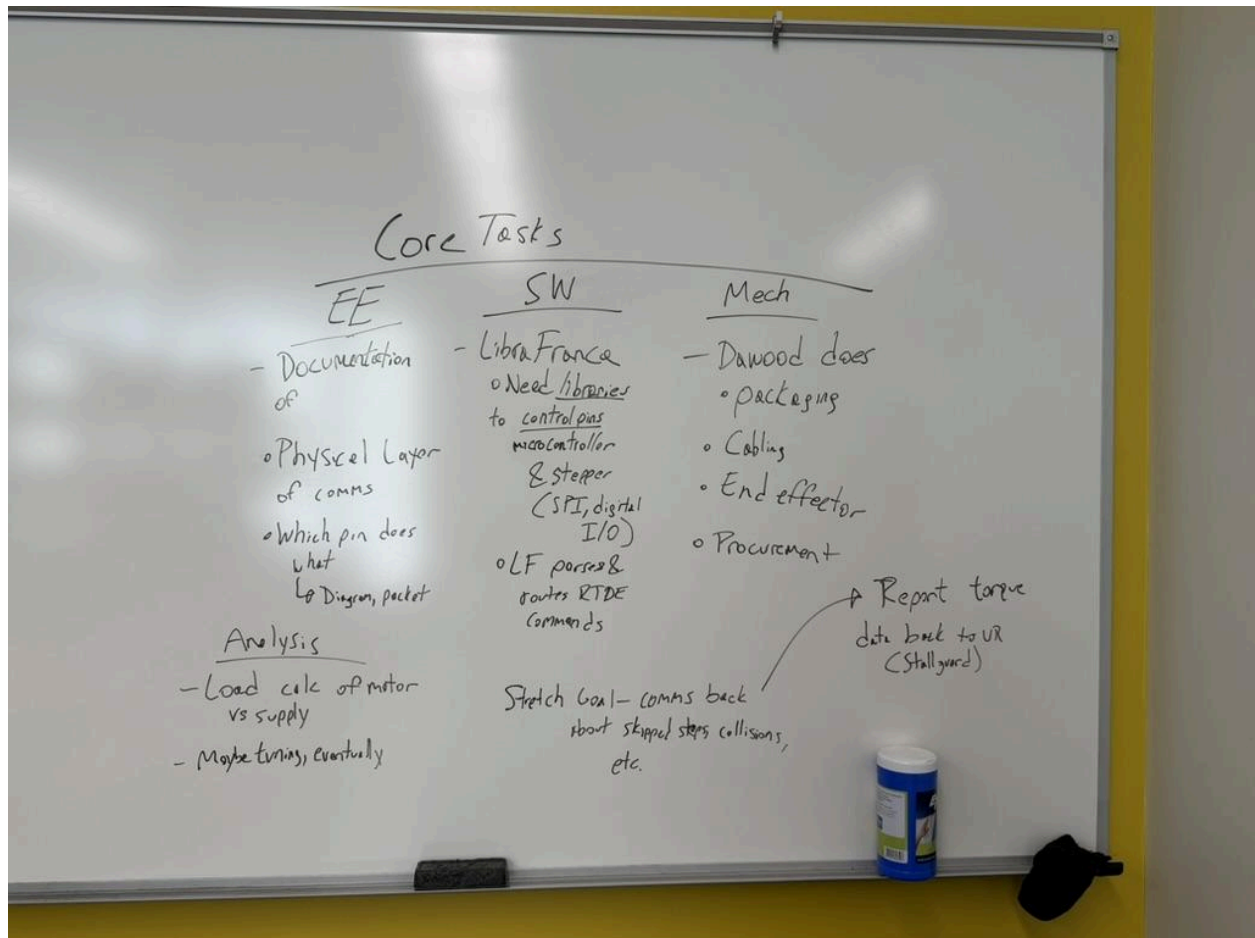
Analysis Steps

- ☐ Load calcs of motor vs supplied (amperage)
- ☐ Maybe eventually tune response
- ☐

Stretch Goals

- ☐ Comms back to URScript about skipped steps, collisions, etc
- ☐ Report torque data back to URScript (Stallguard)
 - ☐ Get it back to the Pi
 - ☐ What does that torque data *mean*?
 - ☐ Potentially, use the torque data down the road to size a gearbox for later features
 - ☐ Stepper gets super hot

Whiteboard Drawing from Meeting



New Stuff!

Original Plan

URScript -> Microcontroller -> Stepper (using RTDE comms)

New Plan

URScript -> **Pi (using Klipper)** -> Microcontroller -> Stepper (using *what?*)

Comms between Pi & URScript (UR Controller / Polyscope)

RTDE currently works using TCP/IP (ethernet connectors)

May need a switch added (any Gigabit network switch will do)

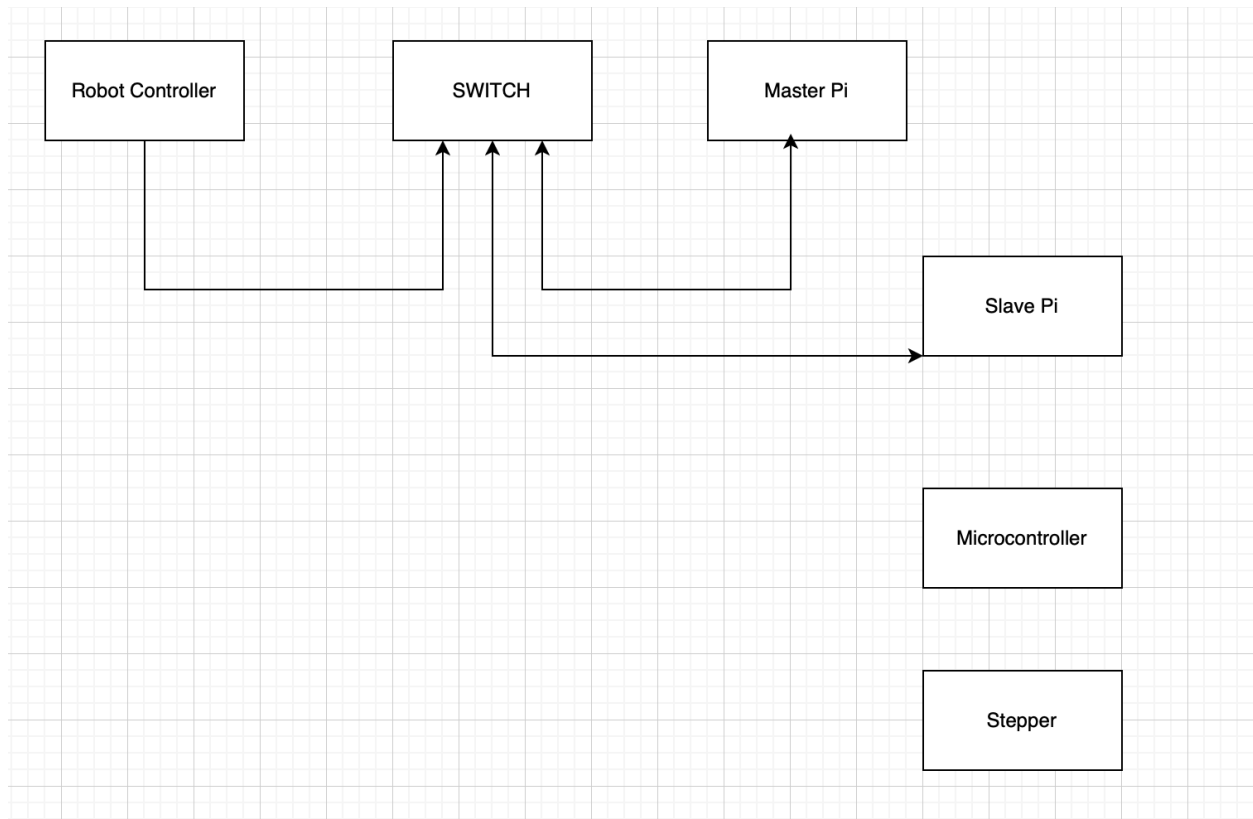
Programmatically, how do URScript & Klipper converse?
Supported protocols by Klipper?

Comms between Pi & Microcontroller
 Serial (via Klipper) - [Klipper](#)

Trade the following protocols (want to send & receive)

Other Open Questions

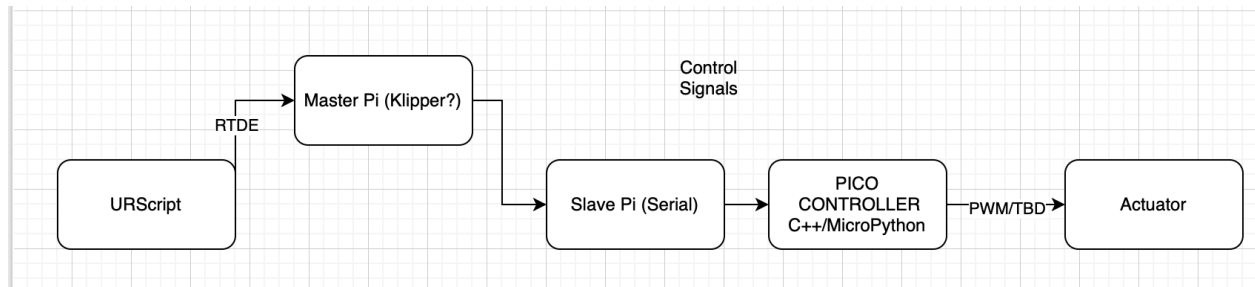
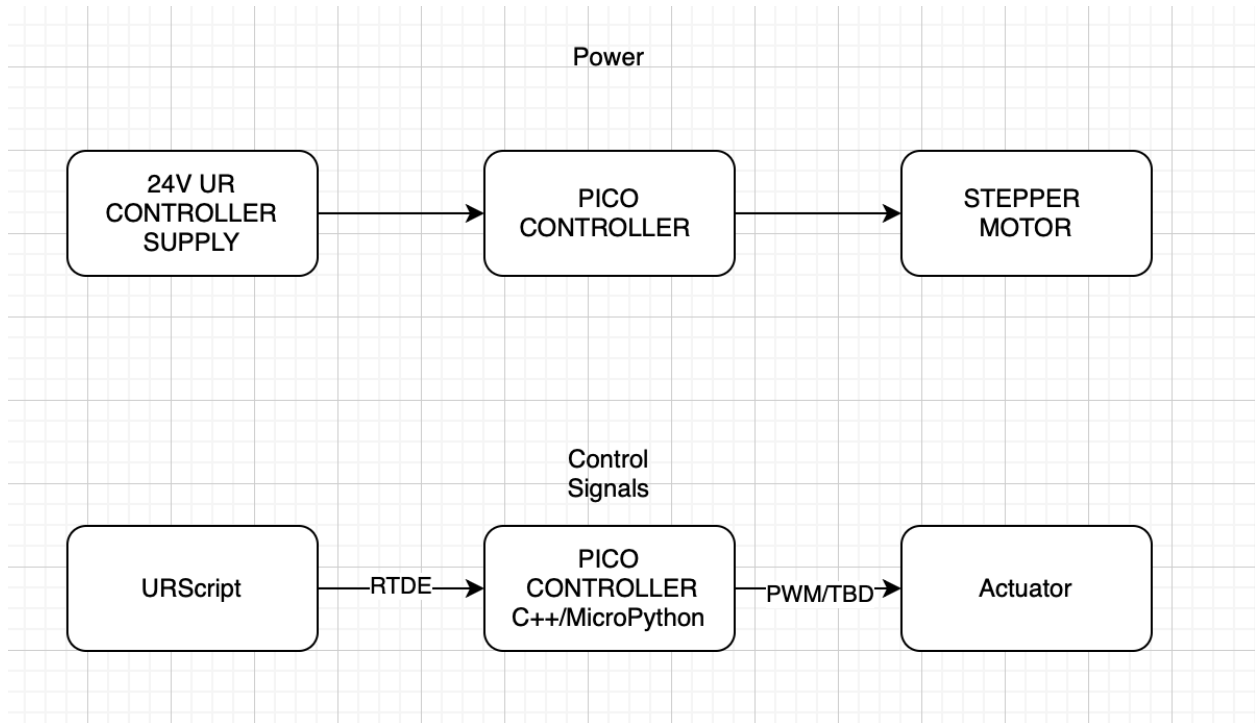
- ☐ Are we adding too much latency via ethernet comms, etc?
 - ☐ Compared to the speed of the print
 - ☐ If it's predictable, can timeshift G-code execution
- ☐ Investigate UR comms protocols & which is the best for our implementation (maybe immediate vs long-term)
 - ☐ How UR CAPs are made
- ☐ Using Klipper
 - ☐ How to send back the collisions, torque data, etc.
 - ☐ Maybe it needs to be a forked feature?
- ☐ Investigation
 - ☐ URScript to Klipper bidirectionality
 - ☐ Klipper functionality vs what we want it to do
 - ☐ Overall, for all features!



Parts

- ☐ We might need
 - ☐ MicroSD cards
 - ☐ Raspberry Pi 400
 - ☐ Cables?
- ☐ Things we have already
 - ☐ Microcontroller
 - ☐ Many Pi's
 - ☐ Stepper

Old Method



Next Week of Tasks

1. Software

- a. Get familiar with Linux/Bash
<https://ubuntu.com/tutorials/command-line-for-beginners#1-overview>
- b. Get familiar with le pi -
<https://www.raspberrypi.com/documentation/computers/getting-started.html>
- c. Get familiar with URScript
 - i. Look at RTDE
- d. Get familiar with Klipper - <https://www.klipper3d.org/>
- e. Get familiar with other comms protocols

- f. Get familiar with Pico controller -
<https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008279-DS-1-hardware-design-with-rp2040.pdf?disposition=inline>
- g. Get familiar with how to drive stepper motors!
 - i. Ex: how to drive microcontroller comms between it and the pi
 EXAMPLE - not how we're doing this, but useful context
<https://learn.adafruit.com/use-dc-stepper-servo-motor-solenoid-rp2040-pico/overview>
- h. Pi400
 - i. Receives commands from RTDE/URController, goes to klipper
 - i. Define latency, write changes to the Klipper later
 - j. Start to have opinions on implementation
- 2. Electrical
 - a. Figure out what devices need what type of power (3.3V, 5V, 24V)
 - b.
- 3. Parts
 - a. What devices do we have presently vs don't?
- 4. Command structure URScript -> Pi ->
- 5. Questions
 - a. Come back with lots of Q's